A Differential Equation Practice Generator

James Cole
Gruff Howell

1 Summary

We made a Python library that creates and solves differential equations so that students can practise. It picks coefficients at random, builds a valid equation, and then gives back a full solution with the working shown step by step. The aim is to give students an easy way to get as many practice problems as they want without having to write them by hand.

2 Statement of Need

When you are learning ODEs you need to do a lot of them before it really sticks. Standard textbooks such as Boyce and DiPrima [6] only have a fixed number of examples and once you have seen them once they are not much use. Tools like SymPy [1] can solve an ODE for you but you have to type the equation in yourself, and Wolfram Alpha [5] is the same. Neither one is built for making practice questions in bulk. Our library fills this gap. You ask it for a question of a certain type and difficulty and it gives you both the question and the worked solution, so you can try it yourself first and then check.

3 Functionality

The library covers seven types of ODE:

- First order linear
- Separable
- Bernoulli
- Homogeneous (using the substitution $z = y/x$)
- Second order with constant coefficients (homogeneous)
- Second order with constant coefficients (inhomogeneous)
- Coupled first order linear ODEs

Each one lives in its own module and has two parts, a generator and a solver. The generator picks random coefficients (with added parameters to match Dr Lydia Buckingham's style) and builds an equation of the right form. The solver does the working using SymPy and returns a list of steps along with the final answer. There is also a function that makes a full problem sheet and can export it as a PDF, which is helpful for revising.

4 How We Solve Each Type

This section walks through what the solver does for each type. We tried to follow the same methods we were taught in lectures.

4.1 First order linear

The standard form is

$$\frac{dy}{dx} + p(x)\,y = q(x).$$

1. Read off $p(x)$ and $q(x)$. If the coefficient of y' is not 1 we divide through first.
2. Work out the integrating factor
$$I(x) = e^{\int p(x)\,dx}.$$
3. Multiply the whole equation by $I(x)$. The left hand side now becomes $\frac{d}{dx}\big[I(x)\,y\big]$, which is just the product rule in reverse.
4. Integrate both sides with respect to x, remembering to add the constant C_1.
5. Divide by $I(x)$ to get y on its own. This is the general solution.
6. If an initial condition is given, plug it in and solve for C_1.

4.2 Separable

The form here is

$$g(y)\,\frac{dy}{dx} = f(x).$$

1. Identify $g(y)$ and $f(x)$.
2. Separate the variables so all the y stuff is on one side and all the x stuff on the other.
3. Integrate each side. This gives an implicit relation between x and y plus a constant C_1.
4. Try to rearrange for y explicitly. Sometimes you can, sometimes you have to leave it implicit.
5. If there is an initial condition, sub it in to find C_1.

4.3 Bernoulli

A Bernoulli equation looks like

$$\frac{dy}{dx} + a(x)\,y = b(x)\,y^n,$$

where n is not 0 or 1 (otherwise it would just be linear).

1. Rearrange into the form above and read off $a(x)$, $b(x)$ and n.
2. Use the substitution $z = y^{1-n}\,e^{(1-n)\int a\,dx}$. Differentiating z and substituting the original ODE makes the y terms cancel.
3. You are left with $\frac{dz}{dx}$ equal to a function of x only, so integrate it.
4. Add the constant C_1 and then sub back $y = z^{1/(1-n)}/e^{\int a\,dx}$ to recover y.
5. Apply the initial condition if there is one.

4.4 Homogeneous ($z = y/x$ substitution)

This is for equations of the form

$$\frac{dy}{dx} = f\Big(\frac{y}{x}\Big).$$

1. Check that the right hand side really is a function of y/x only.
2. Let $z = y/x$ so $y = zx$. By the product rule $\frac{dy}{dx} = x\frac{dz}{dx} + z$.
3. Substitute this in to get $x\frac{dz}{dx} = f(z) - z$, which is now separable.
4. Separate the variables and integrate both sides. The right side gives $\ln|x| + C_1$.
5. Substitute $z = y/x$ back in and rearrange for y if you can.
6. Also check if $f(z) - z = 0$ has any roots, because those give extra constant solutions $y = z_0 x$ that can otherwise be missed.
7. Apply the initial condition.

4.5 Second order, constant coefficients, homogeneous

The form is

$$a\,y'' + b\,y' + c\,y = 0,$$

where a, b, c are constants.

1. Write down the auxiliary equation $a\mu^2 + b\mu + c = 0$ and solve for μ.
2. Look at the discriminant $b^2 - 4ac$ and split into three cases:

- Two real distinct roots μ_1, μ_2:

$$y = A\,e^{\mu_1 x} + B\,e^{\mu_2 x}.$$

- One repeated root μ:

$$y = A\,e^{\mu x} + B\,x\,e^{\mu x}.$$

- Complex roots $p \pm iq$:

$$y = e^{px}\big(A\cos(qx) + B\sin(qx)\big).$$

3. This gives you the general solution with two constants A and B.

4. If you have two initial conditions, sub them in (and differentiate once for y') and solve the resulting system for A and B.

4.6 Second order, constant coefficients, inhomogeneous

Now the form is

$$a\,y'' + b\,y' + c\,y = f(x).$$

1. First find the complementary function y_c by solving the homogeneous version (same method as above).

2. Pick a trial particular solution y_p that matches the type of $f(x)$:

   - If $f(x)$ is a constant, try $y_p = C$.
   - If $f(x)$ is a polynomial of degree n, try a general polynomial of degree n.
   - If $f(x)$ is e^{kx}, try $y_p = C_1 e^{kx} + C_2$.
   - If $f(x)$ is $\sin(\theta x)$ or $\cos(\theta x)$, try $y_p = C_1\cos(\theta x) + C_2\sin(\theta x)$.

3. If your trial overlaps with anything in y_c, multiply it by x (and again by x if it still clashes).

4. Sub y_p into the ODE, expand, and compare coefficients to find the unknown C values. (In our code we plug in numbers for x and solve a small linear system instead, but it gives the same answer.)

5. The general solution is $y = y_c + y_p$.

6. Apply initial conditions if given.

4.7 System of two first order linear ODEs

The form is

$$\frac{du}{dx} = a\,u + b\,v, \qquad \frac{dv}{dx} = c\,u + d\,v.$$

1. Differentiate the first equation once more.
2. Use the second equation to replace $\frac{dv}{dx}$, and rearrange the first equation to write v in terms of u and u'.
3. Sub everything back so that you end up with one second order ODE in u only.
4. Solve that second order ODE using the constant coefficient methods above. This gives u with constants A and B.
5. Recover v from the relation you got in step 2.
6. If initial conditions are given for both $u(0)$ and $v(0)$, sub them in and solve for A and B.

5 Verifying the Answer

After solving, the library checks the answer by plugging it back into the original ODE and seeing if everything cancels to zero. It also checks the initial conditions if there are any. This way the user (and us) can be confident the solver is not making things up. For implicit solutions we use implicit differentiation to recover y' before substituting.

6 State of the Field

Several established systems already deal with differential equations. SymPy [1] is a Python computer algebra library and it is what powers our solver under the hood, since it integrates directly with normal Python code. Other general-purpose computer algebra systems include Maxima [2], the open-source SageMath [3] (which itself bundles SymPy and Maxima among others), and the commercial packages Mathematica [4] and Maple. Online tools such as Wolfram Alpha [5] can solve a given equation on demand but are aimed at one-off lookups rather than at producing batches of fresh practice problems. Textbooks such as Boyce and DiPrima [6] are still the standard reference for an undergraduate course but again only contain a fixed pool of examples.

None of these are set up to act as a self-contained problem generator for a first year student. Our library is much smaller in scope than any of the systems above but fills exactly that gap. In spirit it is similar to the small focused Python tools published in the Journal of Open Source Software, for example Nashpy [9] for Nash equilibria and Matching [10] for matching games, both of which wrap a specific bit of mathematics in a simple educational interface.

7 Testing

We wrote a pytest [8] test suite that checks both the generators and the solvers. The generator tests check that each problem is built correctly and is of the type it claims to be. The solver tests run a sample of problems and verify the answers symbolically. The test file passes 118 tests covering all seven ODE types and the problem sheet exporter. The documentation in `README.md` also contains runnable examples checked with `doctest`, so the examples shown to the user are guaranteed to still produce the output the docs claim.

8 Conclusion

The project shows a working, modular way of generating ODE practice problems with full step by step solutions. It is mostly aimed at first year Cardiff Mathematics students who want extra practice past what their textbook gives them. The documentation follows the Diataxis framework [7] with a Tutorial, How-to, Explanation and Reference section so that a new user can pick up the library quickly. We are likely to open-source this at the end of the year in the event of future first years in need of an infinite supply of problem sheets.

There are still things we would like to improve. The step explanations are short and could be more detailed. We also only cover seven types of ODE, so adding things like exact equations, or higher order systems would be a nice extension (third order ODE's would have been useful to practice before the exam). The frontend is fairly basic and could be made nicer too but as this is not within our skillset for now this is unlikely to be changed any time soon.

References

[1] Meurer, A. et al. *SymPy: symbolic computing in Python*. PeerJ Computer Science, 3:e103, 2017. `https://www.sympy.org`

[2] Maxima Development Team. *Maxima, a Computer Algebra System*. `https://maxima.sourceforge.io`

[3] The Sage Developers. *SageMath, the Sage Mathematics Software System*. `https://www.sagemath.org`

[4] Wolfram Research, Inc. *Mathematica*, Version 13. `https://www.wolfram.com/mathematica`

[5] Wolfram Alpha LLC. *Wolfram Alpha computational engine*. `https://www.wolframalpha.com`

[6] Boyce, W. E. and DiPrima, R. C. *Elementary Differential Equations and Boundary Value Problems*, 11th edition. Wiley, 2017.

[7] Procida, D. *The Diataxis documentation framework*. `https://diataxis.fr`

[8] Krekel, H. et al. *pytest: helps you write better programs*. `https://docs.pytest.org`

[9] Knight, V. *Nashpy: A Python library for the computation of Nash equilibria*. Journal of Open Source Software, 3(30):904, 2018.

[10] Wilde, H. and Knight, V. *Matching: A Python library for solving matching games*. Journal of Open Source Software, 5(48):2169, 2020.